

```

In[7]:= Print["Wolfram Language ready!"]
Print[$Version]

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)

In[38]:= (* ===== Neuron Parameters ===== *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ===== Input Current ===== *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."]

Parameters defined.

Knowledge Check

```

1. The input current is defined as a function of time using `If[]`. In plain English, what does `Iinput[t]` return at $t = 0.03$? At $t = 0.2$? At $t = 0.5$?

At $t = 0.03$ the value returned is 0 because the condition $0.05 \leq t \leq 0.4$ is not satisfied yet. This means the current has not turned on.

At $t = 0.2$ the value returned is 18.0 because the time falls within the interval where the input current is active.

At $t = 0.5$ the value returned is 0 again because the time is past 0.4 seconds, which means the current has turned off.

2. Why do we use `:=` (delayed assignment) for `Iinput` instead of `=` (immediate assignment)?

We use delayed assignment so that the function evaluates every time the variable `t` changes during the simulation. If we used immediate assignment, the expression would be evaluated once and stored as a fixed value instead of updating dynamically as time changes.

```
In[45]:= spikeTimes = {};
```

```
sol = NDSolve[
{
  tau * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.4, 18.0, 0.0],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh,
    {V[t] → Vreset,
     AppendTo[spikeTimes, t]}]
},
V,
{t, 0, 0.5}
];
```

```
spikeCount = Length[spikeTimes];
Print["Neuron fired ", spikeCount, " spikes"]
Print["Spike times: ", spikeTimes]

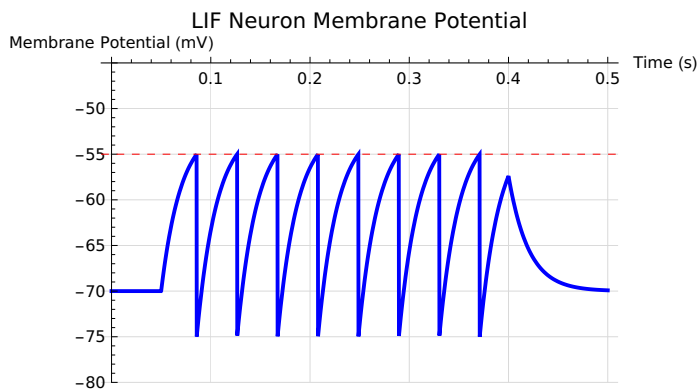
Neuron fired 8 spikes

Spike times:
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

```

In[50]:= Plot[
  V[t] /. sol,
  {t, 0, 0.5},
  PlotLabel → "LIF Neuron Membrane Potential",
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotRange → {-80, -45},
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
  PlotStyle → Blue,
  GridLines → Automatic,
  GridLinesStyle → LightGray
]
Out[50]=

```



Knowledge Check

1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?

The shape of the membrane potential between spikes is a curve rather than a straight line. This happens because the LIF model follows a differential equation that produces exponential behavior. The voltage gradually approaches the threshold as the neuron integrates the input current.

2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

After the current turns off at $t > 0.4$, the membrane potential slowly moves back toward the resting potential. This happens because the equation includes the term $-(V[t] - V_{\text{rest}})$, which causes the voltage to decay back toward the resting value when there is no input current.

In[52]:=

```

Manipulate[
Module[{sol, st = {}, sc},

sol = NDSolve[
{
tauVal * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.4, currentVal, 0.0],
V[0] == Vrest,
WhenEvent[V[t] ≥ thresh,
{V[t] → Vreset,
AppendTo[st, t]}]
},
V,
{t, 0, 0.5}
];

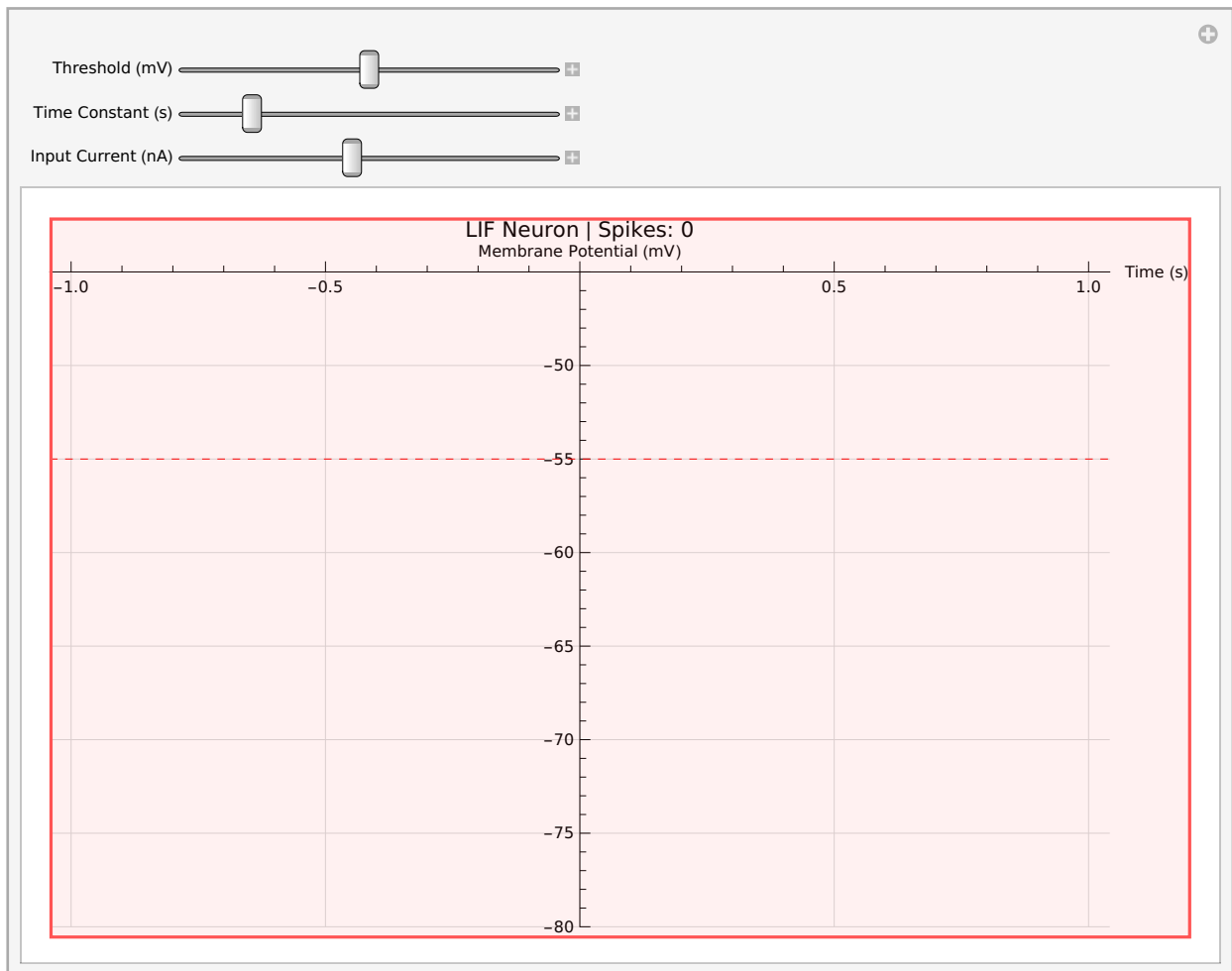
sc = Length[st];

Plot[
V[t] /. sol,
{t, 0, 0.5},
PlotLabel → StringForm["LIF Neuron | Spikes: ``", sc],
AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
PlotRange → {-80, -45},
Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]},
PlotStyle → Blue,
GridLines → Automatic,
GridLinesStyle → LightGray,
ImageSize → 600
]
],

{{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
{{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
{{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]

```

Out[52]=



Knowledge Check

1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

The minimum input current required for the neuron to fire at least one spike was around 16–18 nA. When the current was below this value, the membrane potential did not reach the firing threshold.

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

When the threshold is gradually increased, the neuron eventually stops firing at around -48 mV to -50 mV. This occurs because the membrane potential is no longer able to reach the higher threshold value before the input current ends.

3. Set threshold to -55 and current to 20. Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more

spikes? Which produces a smoother membrane potential curve?

With $\tau = 0.010$ the neuron produces more spikes because the membrane responds faster to input changes. With $\tau = 0.050$ the membrane potential curve appears smoother and the neuron fires fewer spikes because the voltage changes more slowly.

Reflection

The Manipulate dashboard allowed me to explore how different neuron parameters influence firing behavior. By adjusting the sliders for threshold, time constant, and input current, I could immediately see how the firing rate changed. One interesting insight was how sensitive the neuron was to small changes in threshold. Even a small increase in threshold could stop the neuron from firing entirely. The visual feedback made it much easier to understand how neurons integrate inputs and generate spikes compared to only reading about the model.

In[53]:=

```

fiData = Table[
Module[{st = {}, solFI, sc},
solFI = NDSolve[
{
tau * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.45, Icurr, 0.0],
V[0] == Vrest,
WhenEvent[V[t] ≥ Vthresh,
{V[t] → Vreset,
AppendTo[st, t]}]
},
V,
{t, 0, 0.5}
];
sc = Length[st];
{Icurr, sc}
],
{Icurr, 0, 35, 1}
];

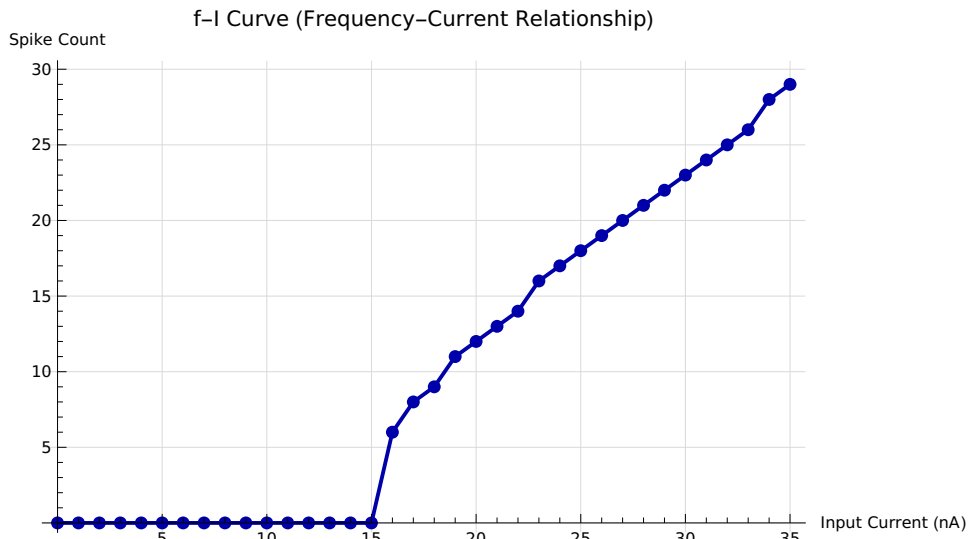
```

```

ListLinePlot[
fiData,
AxesLabel → {"Input Current (nA)", "Spike Count"},
PlotLabel → "f-I Curve (Frequency-Current Relationship)",
PlotMarkers → Automatic,
PlotStyle → Darker[Blue],
GridLines → Automatic,
GridLinesStyle → LightGray,
ImageSize → 500
]

```

Out[54]=



Knowledge Check

1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

The neuron first started firing at 17 nA of input current, which matches the rheobase value observed using the interactive dashboard.

2. Is the f–I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

The f–I curve is not perfectly straight. It begins flat when the current is too small to generate spikes and then increases as the current becomes stronger. A perfectly straight line would mean that the firing rate increases proportionally with stimulus intensity.

3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

The basic LIF model does not strongly show saturation. Real neurons have biological limits such as refractory periods and ion channel dynamics that limit firing rate, but these mechanisms are not fully included in the simplified LIF model.

```
In[55]:= Isine[t_] := 15.0 + 12.0 * Sin[2 Pi * 5 * t];
```

```
spikeTimesSine = {};
```



```

solSine = NDSolve[
{
  tau * V'[t] == -(V[t] - Vrest) + R * Isine[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh,
    {V[t] → Vreset,
     AppendTo[spikeTimesSine, t]}]
},
V,
{t, 0, 1.0}
];

spikeCountSine = Length[spikeTimesSine];
Print["Sine wave spikes: ", spikeCountSine]

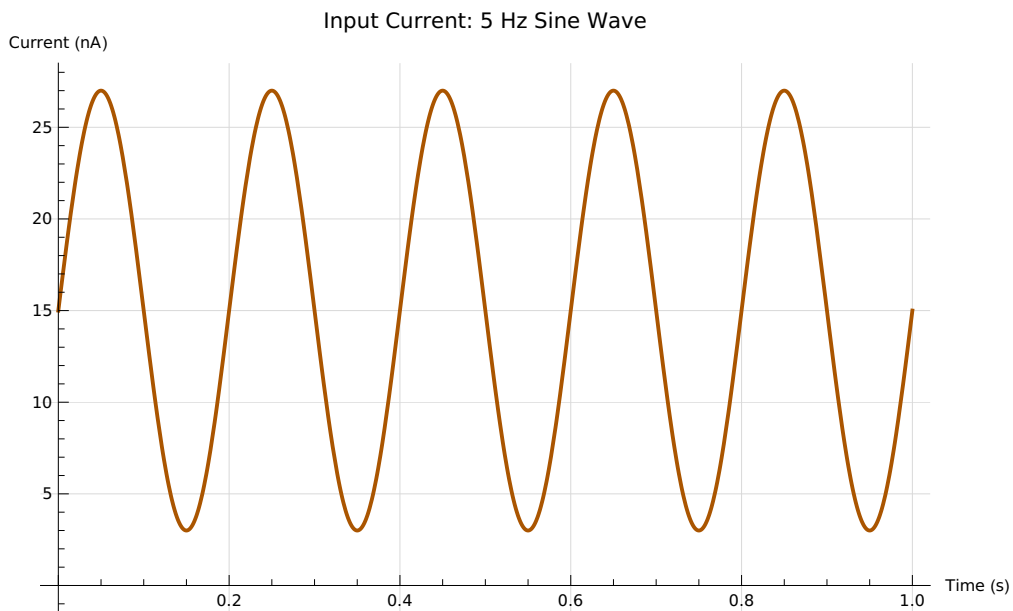
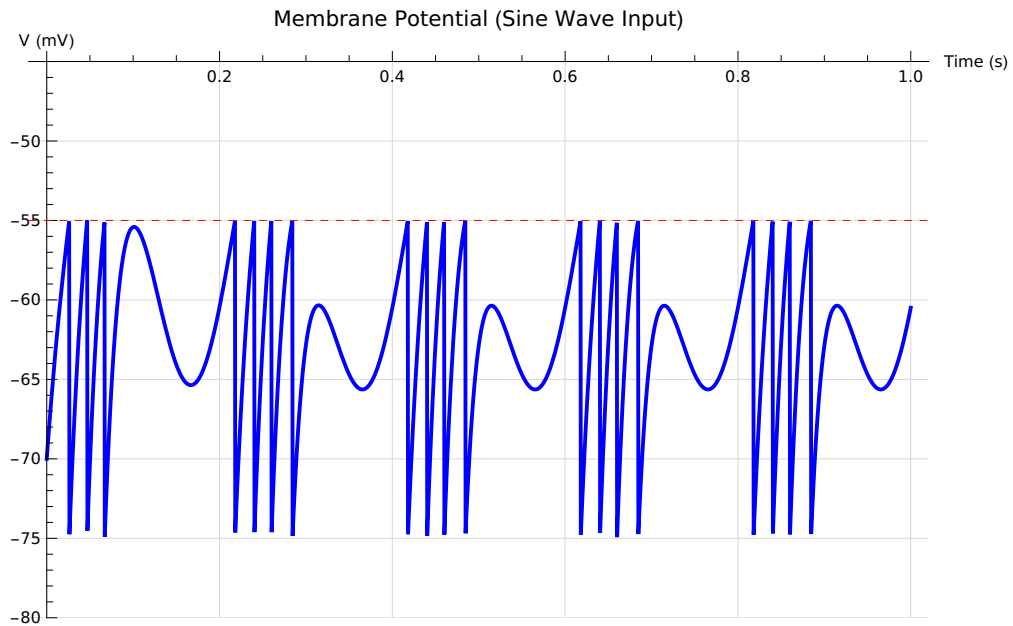
topPlot = Plot[
  V[t] /. solSine,
  {t, 0, 1.0},
  PlotLabel → "Membrane Potential (Sine Wave Input)",
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotRange → {-80, -45},
  Epilog → {Red, Dashed, InfiniteLine[{0, Vthresh}, {1, Vthresh}]},
  PlotStyle → Blue,
  GridLines → Automatic,
  GridLinesStyle → LightGray
];

bottomPlot = Plot[
  Isine[t],
  {t, 0, 1.0},
  PlotLabel → "Input Current: 5 Hz Sine Wave",
  AxesLabel → {"Time (s)", "Current (nA)"},
  PlotStyle → Darker[Orange],
  GridLines → Automatic,
  GridLinesStyle → LightGray
];

```

```
GraphicsColumn[{topPlot, bottomPlot}, Spacings -> 20]
```

Sine wave spikes: 19



Reflection

Look at the spike train alongside the sine wave input. The neuron acts like an analog-to-digital converter by transforming a continuous signal into discrete spikes. The spike density tends to increase

when the sine wave reaches its peaks and decrease when the signal is low. This means the neuron preserves information about the strength and timing of the input signal. However, the exact shape of the sine wave is not preserved because the output only contains spikes rather than a continuous waveform.

```
In[63]:= threshSweepData = Table[
  Module[{st = {}, solTh, sc},
    solTh = NDSolve[
      {
        tau * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.4, 18.0, 0.0],
        V[0] == Vrest,
        WhenEvent[V[t] ≥ thVal,
          {V[t] → Vreset, AppendTo[st, t]}]
      },
      V, {t, 0, 0.5}
    ];
    sc = Length[st];
    {thVal, sc}
  ],
  {thVal, -65, -45, 5}
];
```

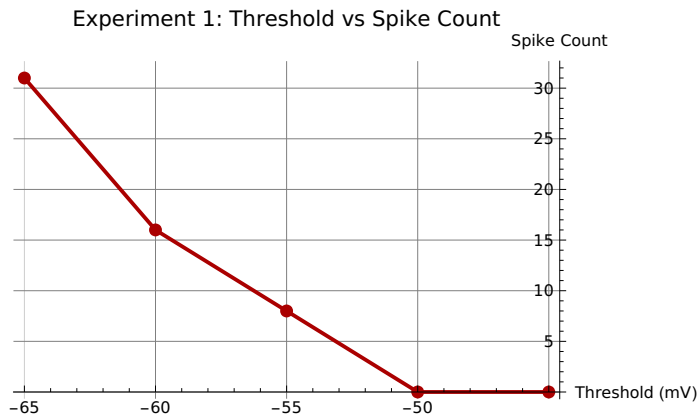
```
Print["Threshold (mV) vs Spike Count:"]
Print[TableForm[threshSweepData,
  TableHeadings → {None, {"Threshold (mV)", "Spike Count"}}]]
```

```
ListLinePlot[
  threshSweepData,
  AxesLabel → {"Threshold (mV)", "Spike Count"},
  PlotLabel → "Experiment 1: Threshold vs Spike Count",
  PlotMarkers → Automatic,
  PlotStyle → Darker[Red],
  GridLines → Automatic
]
```

Threshold (mV) vs Spike Count:

Threshold (mV)	Spike Count
-65	31
-60	16
-55	8
-50	0
-45	0

Out[66]=



Experiment 1: Threshold Sweep

1. What parameter did you change, and to what values?

I changed the firing threshold of the neuron from -65 mV to -45 mV in steps of about 5 mV while keeping the other parameters constant.

2. What did you expect to happen?

I expected that a lower threshold would make the neuron fire more spikes, because the membrane potential would reach the threshold more easily.

3. What actually happened?

The neuron fired many spikes when the threshold was lower (closer to -65 mV). As the threshold increased toward -45 mV, the number of spikes decreased and eventually the neuron stopped firing.

4. Why does this make sense given the LIF equation?

In the LIF model, the neuron fires when the membrane potential reaches the threshold. A higher threshold requires more input current to reach that level, so it becomes harder for the neuron to fire spikes.

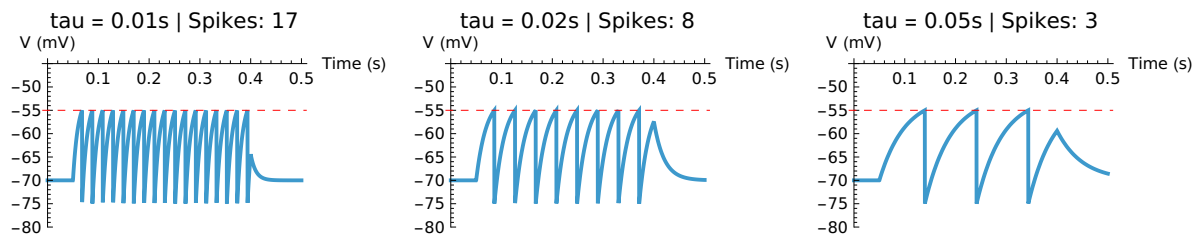
```

In[67]:= tauPlots = Table[
  Module[{st = {}, solTau, sc},
    solTau = NDSolve[
      {
        tauVal * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.4, 18.0, 0.0],
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vthresh,
          {V[t] → Vreset, AppendTo[st, t]}]
      },
      V, {t, 0, 0.5}
    ];
    sc = Length[st];
    Plot[
      V[t] /. solTau,
      {t, 0, 0.5},
      PlotLabel → StringForm["tau = `s | Spikes: `", tauVal, sc],
      AxesLabel → {"Time (s)", "V (mV)"},
      PlotRange → {-80, -45},
      Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
      ImageSize → 280
    ]
  ],
  {tauVal, {0.010, 0.020, 0.050}}
];

```

GraphicsRow[tauPlots, Spacings → 10]

Out[68]=



Experiment 2: Time Constant Comparison

1. What parameter did you change, and to what values?

I compared three values for the membrane time constant: $\tau = 0.010$, $\tau = 0.020$, and $\tau = 0.050$.

2. What did you expect to happen?

I expected that a smaller time constant would allow the neuron to respond faster to the input current and potentially produce more spikes.

3. What actually happened?

When τ was smaller, the membrane potential increased more quickly and the neuron fired more frequently. When τ was larger, the voltage changed more slowly and the membrane potential curve appeared smoother.

4. Why does this make sense given the LIF equation?

The time constant controls how quickly the membrane potential changes in response to input current. A smaller τ means the neuron reacts faster, while a larger τ means the voltage integrates input more slowly.

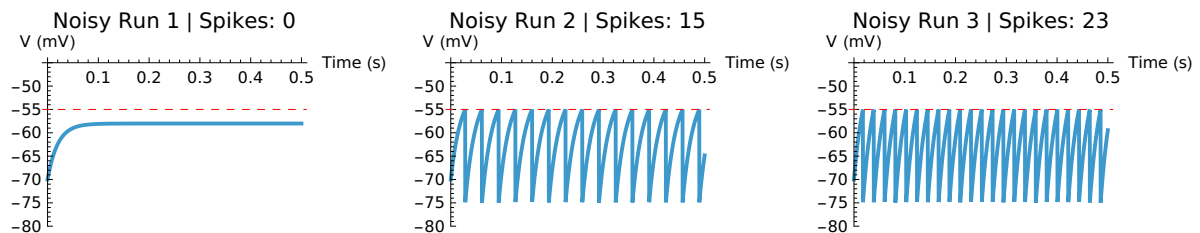
```

In[69]:= noisyPlots = Table[
  Module[{st = {}, solNoisy, sc},
    solNoisy = NDSolve[
      {
        tau * V'[t] == -(V[t] - Vrest) + R * (18.0 + RandomReal[{-8, 8}]),
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vthresh,
          {V[t] → Vreset, AppendTo[st, t]}]
      },
      V, {t, 0, 0.5}
    ];
    sc = Length[st];
    Plot[
      V[t] /. solNoisy,
      {t, 0, 0.5},
      PlotLabel → StringForm["Noisy Run `` | Spikes: ``", run, sc],
      AxesLabel → {"Time (s)", "V (mV)"},
      PlotRange → {-80, -45},
      Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
      ImageSize → 280
    ]
  ],
  {run, 1, 3}
];

```

```
GraphicsRow[noisyPlots, Spacings → 10]
```

Out[70]=



Experiment 3: Noisy Input

1. What parameter did you change, and to what values?

I replaced the constant input current with a noisy input current using the function $I_{\text{noisy}}[t] := 18.0 + \text{RandomReal}[\{-8, 8\}]$.

2. What did you expect to happen?

I expected that the spike pattern would become less predictable because the input current was changing randomly.

3. What actually happened?

Each time the simulation ran, the spike pattern was slightly different. Sometimes the neuron fired more spikes and sometimes fewer spikes depending on the random values generated.

4. Why does this make sense given the LIF equation?

The membrane potential depends directly on the input current. When the input current fluctuates randomly, the membrane potential also fluctuates, which changes when the threshold is reached and therefore changes the spike timing.


```
In[78]:= spikeTimesRef = {};
```

```
solRef = NDSolve[
{
  tau * V'[t] == -(V[t] - Vrest) + R * If[0.05 ≤ t ≤ 0.4, 18.0, 0.0],
  V[0] == Vrest,
  WhenEvent[
    V[t] ≥ Vthresh && (Length[spikeTimesRef] == 0 || t - Last[spikeTimesRef] > 0.003),
    {V[t] → Vreset,
     AppendTo[spikeTimesRef, t]}
  ]
},
V,
{t, 0, 0.5}
];
```

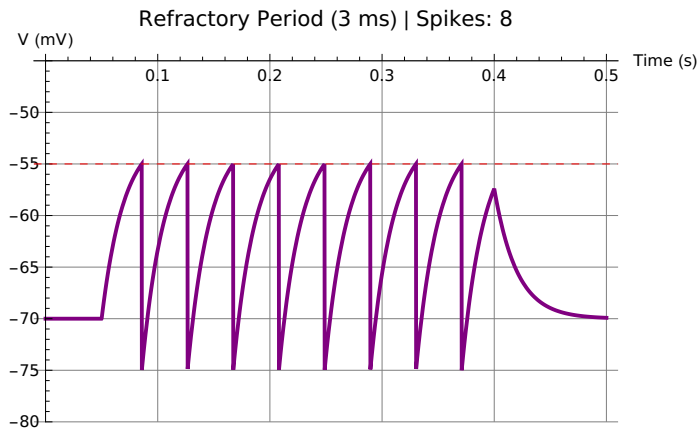
```
spikeCountRef = Length[spikeTimesRef];
Print["With refractory period: ", spikeCountRef, " spikes"]
Print["Without refractory (from Part A): ", spikeCount, " spikes"]
```

```
Plot[
  V[t] /. solRef,
  {t, 0, 0.5},
  PlotLabel → StringForm["Refractory Period (3 ms) | Spikes: ``", spikeCountRef],
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotRange → {-80, -45},
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
  PlotStyle → Purple,
  GridLines → Automatic
]
```

```
With refractory period: 8 spikes
```

```
Without refractory (from Part A): 8 spikes
```

Out[83]=



Experiment 4: Refractory Period

1. What parameter did you change, and to what values?

In this experiment I added a refractory period to the neuron model. This means that after a spike occurs, the neuron must wait at least 3 milliseconds (0.003 seconds) before it can fire another spike.

2. What did you expect to happen?

I expected that adding a refractory period would reduce the maximum firing rate of the neuron, because the neuron would not be able to fire again immediately after a spike.

3. What actually happened?

The spike count remained the same at 8 spikes, identical to the result from Part A without a refractory period. The spike timing did not change significantly between the two versions.

4. Why does this make sense given the LIF equation?

This makes sense because with an input current of 18 nA and the default parameters, the natural interval between spikes is already longer than 3 milliseconds. This means the refractory period was never a limiting factor — the neuron was not firing fast enough for the restriction to matter. A refractory period would only reduce the spike count if the input current were strong enough to drive firing at intervals shorter than 3 ms. This shows that a refractory period only meaningfully constrains firing rate when the stimulus is strong enough to push the neuron to its limits.

Final Reflection

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this

threshold behavior tell you about how biological neurons make “decisions”?

As the input current increases from below threshold, the neuron initially does not fire any spikes. Once the current reaches the rheobase, the membrane potential crosses the threshold and spikes begin to appear. This threshold behavior shows how neurons act like decision-making systems, where inputs must accumulate enough strength before triggering an action potential.

2. Examine your $f-I$ curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

The relationship is not perfectly linear. The curve starts flat when the current is too small to produce spikes and then increases as the current becomes stronger. This demonstrates how neurons encode stronger stimuli by increasing their firing rate.

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

A smaller time constant allows the neuron to respond more quickly to input and generate spikes faster. A larger time constant causes the voltage to change more slowly and produces fewer spikes. Neurons with short time constants are better at detecting rapid changes in signals, while neurons with longer time constants integrate information over longer time periods.

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss?

The LIF model captures the basic ideas of membrane potential integration, threshold firing, and spike resets. However, it does not include many biological details such as dendritic structures, multiple ion channels, neurotransmitter effects, or adaptive firing behavior.

5. Wolfram’s Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

Using Manipulate allowed me to interact with the neuron model directly and observe how different parameters affect firing behavior. This made the learning experience more intuitive than just reading theoretical explanations. The most surprising observation was how small changes in parameters like threshold or input current could completely change whether the neuron fired or remained silent.